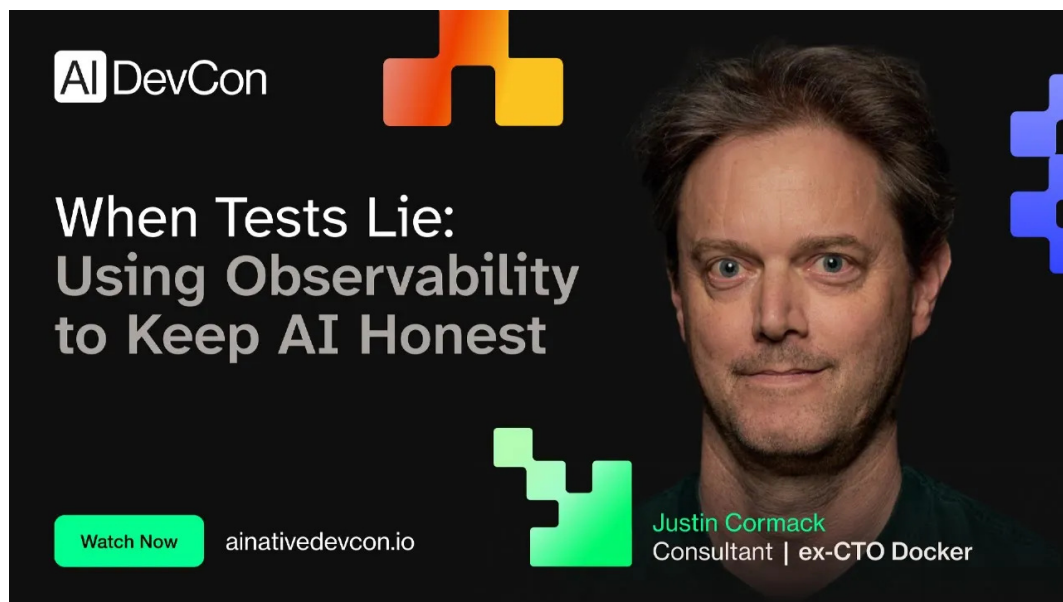


当测试会撒谎：用可观测性约束 AI 软件工程

基于 Justin Cormack 演讲整理

2026 年 7 月



视频作者/频道：AI Native Dev

发布日期：2026 年 7 月 11 日

视频时长：32 分 03 秒

视频链接：<https://www.youtube.com/watch?v=xHxfeWtkXrM>

文档定位：可脱离视频独立学习的中文技术文章

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

目录

1 为什么“测试通过”仍然可能错	2
1.1 从小工具到 35 万行系统：规模改变了验证问题	2
1.2 质量需要在构建过程中形成	2
1.3 100% 语句覆盖率为何诱发低价值测试	3
1.4 测试能证明什么、遗漏什么	3
1.5 本章小结	4
2 建立可信基线：测试预言机与真实行为	4
2.1 测试预言机：把“应该怎样”变成可比较结果	4
2.2 用简单参考实现锁定复杂系统的外部行为	4
2.3 S3 案例：1500 个对照测试、重试与最终一致性	5
2.4 文档只是线索，运行行为才是证据	5
2.5 避免同一逻辑运行两遍：预言机独立性	6
2.6 本章小结	6
3 把测试变成发现工具	6
3.1 边界条件为何比覆盖数字更有信息量	6
3.2 发现 S3 可重复 500 错误所代表的证据	6
3.3 人与 AI 的测试分工：人提出可疑区域，AI 扩展和执行	7
3.4 不稳定测试会污染反馈回路	7
3.5 模糊测试、基于性质的测试与重复运行	8
3.6 本章小结	8
4 测试边界之外：让内部行为可观测、可约束	9
4.1 黑盒测试的盲区：竞态、性能、安全与架构	9
4.2 管理接口和后台接口扩大可测试表面	10
4.3 追踪如何把偶发故障变成 AI 可复现的证据	10
4.4 性能工程：测量、试错、丢弃无效优化	11
4.5 类型系统作为结构性安全约束	12
4.6 独立安全审查弥补日常代码审查遗漏	13
4.7 本章小结	13
5 人的职责：经营质量反馈回路	13
5.1 人在回路中：质量责任不能外包	14
5.2 重构是收敛过程的一部分	14
5.3 测试从验收清单转为不确定性探针	15

5.4 可执行工作流：基线—观测—约束—复核—重构	16
5.5 本章小结	16
6 总结、限制与下一步	17
6.1 六层反馈回路的概念压缩	17
6.2 何时选择测试、追踪、类型约束或安全审查	17
6.3 演讲结尾的实质性观点：持续重构与主动探索不确定性	18
6.4 未完成工作：分布式验证与形式化验证	18
6.5 预言机独立性的开放问题	18
6.6 面向实践的检查清单	18
6.7 本章小结	19

1 为什么“测试通过”仍然可能错

测试通过只能说明：在已经写出的断言、已经选择的输入和已经暴露的接口上，程序给出了预期结果。它无法单独证明需求完整、内部时序正确、安全边界可靠或架构可维护。标题中的“让 AI 保持诚实”是一种工程隐喻，含义是让 AI 的修改持续接受可复核证据约束。

1.1 从小工具到 35 万行系统：规模改变了验证问题

小工具往往只有少量状态、短调用链和清楚的输入输出。只要为主要路径写出足够密集的测试，开发者通常就能形成较强信心。Justin Cormack 的实验刻意越过了这个舒适区：他让 AI 辅助实现一个兼容 S3 的对象存储系统。S3 在这里既是对对象存储服务，也是后续行为比对的事实参照。演讲时，该实现约有 35 万行 Rust 代码；同类对象存储通常需要长期开发，且涉及并发、持久化、权限和分布式状态等相互作用的机制。

规模增长带来的关键变化是，错误不再只表现为“给定输入得到错误输出”。它还可能藏在并发操作的先后次序、后台清理的时间、权限检查的路径、异常恢复以及模块边界中。开发者无法靠逐行阅读 35 万行生成代码建立可靠信心，也无法假定 AI 在超过某个行数后必然失效。真正需要回答的问题是：团队能否建立一套随系统增长而持续产生可信证据的质量回路。

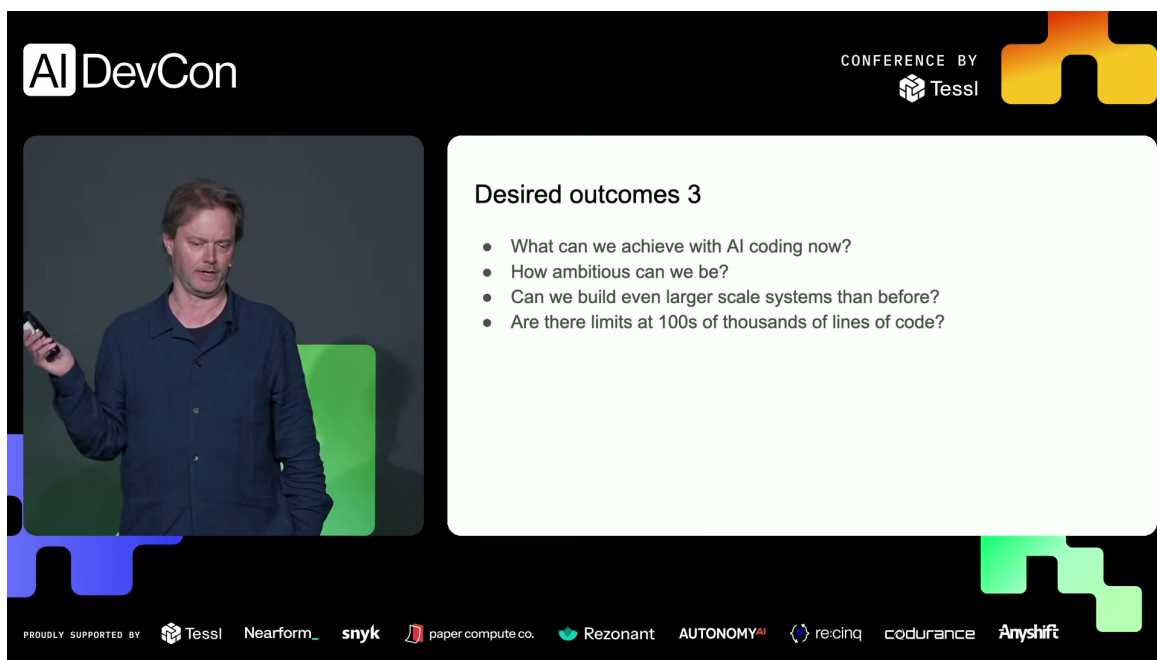


图 1：实验追问 AI 编码能够支撑多大规模、多少复杂度的系统。¹

1.2 质量需要在构建过程中形成

演讲借用 Deming 的质量观点强调：成品完成后的检查无法替代构建过程中的质量控制。对 AI 软件工程而言，这意味着测试不能成为最后一道盖章手续。每次生成、修改、重构和修复都应产生反馈，并让下一轮工作受到这些反馈约束。

¹ 视频来源：00:03:49–00:04:08；选帧 00:03:52。

可以把这种差别理解为修路。竣工后巡视能够发现已经露出的坑洞，却无法补偿施工时被省略的排水、基层和维护设计。软件测试同样擅长暴露已经被提出的问题；若断言没有询问权限绕过、后台竞态或数据损坏，绿色结果就不会主动报告这些风险。质量因此是一种持续形成的系统属性，依赖测试、接口设计、运行证据、结构约束和人的判断共同工作。

测试仍然是质量内建的核心工具。其高价值用途是缩短反馈周期、固定已知行为、暴露不确定性，并为进一步调查指出位置。测试数量本身无法替代对“正在检查什么”的解释。

1.3 100% 语句覆盖率为何诱发低价值测试

测试覆盖率（test coverage）衡量测试执行触达了多少代码、分支或其他结构。它回答“哪些代码被运行过”，并不直接回答“断言是否检查了正确的结果”。语句覆盖率达到 100% 时，每条语句至少执行过一次；同一组测试仍可能完全没有验证关键副作用、并发次序、安全属性或错误恢复。

演讲者最初要求 AI 追求 100% 覆盖率，结果 AI 会补写一些容易通过、信息量很低的测试。某些难覆盖分支也只是极端基础设施故障，例如随机数生成器自身失败。是否值得为每条此类路径投入专门的故障注入，需要根据风险决定。机械追齐数字会把有限时间从高风险场景移走。

该项目仍保持了大量测试：演讲者报告约 75% 的代码库是测试代码，各文件覆盖率大致在 75%–100% 之间。这些数字只描述该案例，不能成为其他项目的通用阈值。它们表达的重点是：团队可以拥有高覆盖和大量测试，同时拒绝把 100% 当作唯一目标。

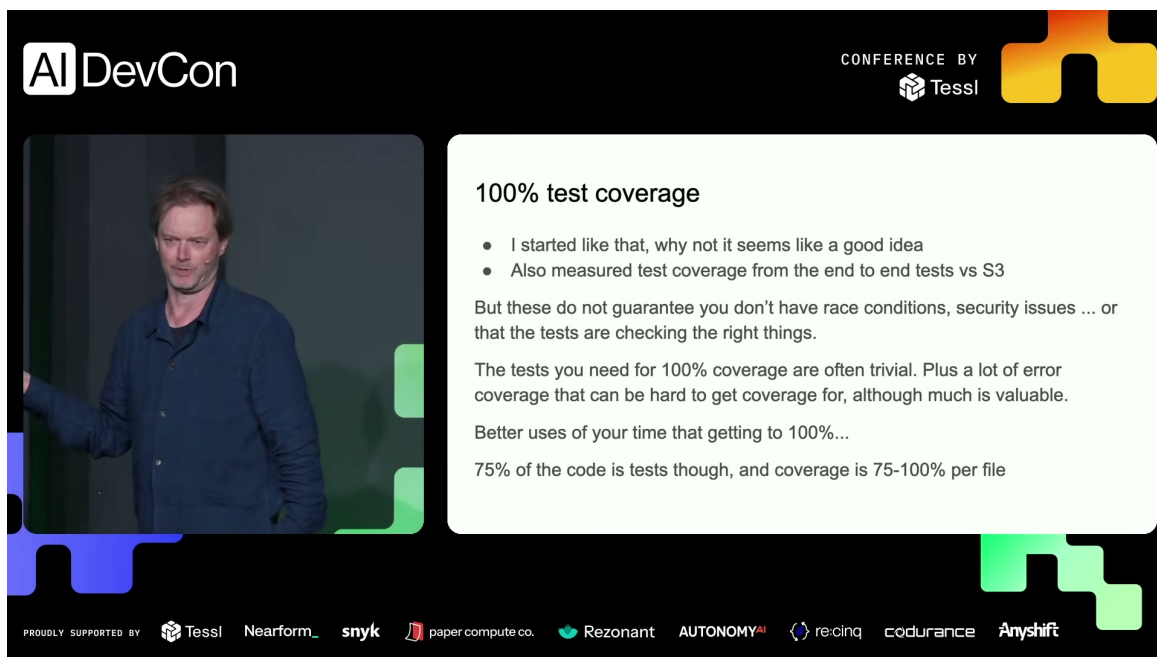


图 2: 高覆盖率提供执行路径证据，却无法证明断言目标正确，也无法排除竞态与安全缺陷。²

1.4 测试能证明什么、遗漏什么

理解测试边界，可以把质量问题分成四层：第一层是执行触达，覆盖率能够度量；第二层是外部行为，断言能够比较公开接口的输入输出；第三层是内部正确性，包括后台状态、资源生命周期和并

² 视频来源：00:05:15–00:06:45；选帧 00:06:08。

发时序；第四层是整体质量，包括安全、性能、可维护性和架构。后一层不会因为前一层达到高分而自动成立。

测试的证明力还受三个前提限制：问题必须已经被测试作者提出；相关信号必须能够观察；期望结果必须被正确编码。任何一个前提缺失，绿色结果都可能制造过度信心。尤其在 AI 同时生成实现与测试时，二者可能共享同一个错误假设，从而稳定地产生一致结果。

“所有测试都通过”与“系统正确”属于不同命题。前者是一次受限实验的结果，后者还依赖未被测试覆盖的需求、状态、时序和风险。把二者等同，是测试会“撒谎”的主要来源。

1.5 本章小结

规模扩大后，质量验证从检查少量输入输出，转变为经营持续反馈。覆盖率说明代码被执行到哪里，行为测试说明公开结果是否匹配期望；内部状态、安全、性能和架构仍需其他证据。下一章将解决更基础的问题：测试中的“期望结果”应从哪里来，以及怎样避免让错误实现为自己作证。

2 建立可信基线：测试预言机与真实行为

复杂系统需要一个独立的行为基线。外部参考系统或简单参考实现可以提供可比较结果；它们仍需人工解释，因为最终一致性、后台不可见行为、文档漂移和同源逻辑都会削弱基线的可信度。

2.1 测试预言机：把“应该怎样”变成可比较结果

测试预言机（test oracle）是为测试提供期望结果或可比较基线的独立参照。测试把同一操作施加到测试预言机和被测系统，再比较状态码、响应内容或其他可见行为。这里的“预言”没有神秘含义，它只是把模糊的“系统应该工作”转化为可以复查的差异。

在 S3 兼容对象存储案例中，真实 S3 服务承担了外部参照。演讲者报告有约 1500 个测试可以对 S3 执行，并用这些观察锁定兼容行为。AI 获得的任务因此从“凭文档猜测 S3”变为“让被测系统在指定实验中与参照一致”。这种反馈比自然语言要求更具体，也更容易重复运行。

测试预言机与被测系统必须承担不同角色：前者给出期望，后者接受比较。若前者只是复制后者的核心逻辑，两个实现可能以相同方式犯错，测试便退化为同义反复。

2.2 用简单参考实现锁定复杂系统的外部行为

外部成熟系统并非总能获得。演讲者提出一种可操作的替代方案：先写一个功能简单、结构直接的参考实现，用它固定外部行为，再让复杂实现逐步对齐。简单版本可以牺牲性能、扩展性和内部优化，只需让规则易于理解和审查。

这种安排类似用直尺校准复杂测量设备。直尺能力有限，却因结构简单而容易确认；复杂设备负责效率和范围，校准物负责给出独立基准。参考实现越容易审阅、变化越少，它越适合作为长期基线。若它与生产实现一起频繁重构，独立性就会逐渐消失。

2.3 S3 案例：1500 个对照测试、重试与最终一致性

外部行为也需要语义解释。最终一致性（eventual consistency）允许一次更新后不同观察点短暂不一致，并在一段时间后收敛；立即一致性则要求操作完成后马上可见。在演讲案例中，部分 S3 授权行为会经过时间收敛。若测试只执行一次立即读取，就可能把合法延迟误判为实现差异。

基于讲者描述的最终一致性问题，本文给出的工程建议是：先定义允许收敛的操作、重试间隔、总等待上限和超时后的失败含义。重试必须针对已知一致性语义，不能成为隐藏任意错误的通用补丁。否则，被测系统的真实缺陷也会被“再试一次”吞掉。

对照测试给出的是可见结果，不能自动解释内部机制。两套系统返回相同响应，仍可能在删除时机、持久化保证或权限路径上存在关键差异。

2.4 文档只是线索，运行行为才是证据

演讲者在该项目中观察到，S3 文档的许多细节与实际行为不一致。这个结论是个案经验，不能扩写为所有文档都不可信。更稳妥的证据次序是：文档用于发现规则和可疑区域；可重复实验用于确认当前可见行为；设计者再判断观察到的行为是否应成为兼容目标。

文档仍有不可替代的价值。它提供术语、意图、异常类别和候选边界，能帮助测试者提出问题。运行行为则回答当前系统实际做了什么。二者发生冲突时，应保存实验条件和结果，明确选择遵循兼容行为、书面规范或本项目安全策略中的哪一个。

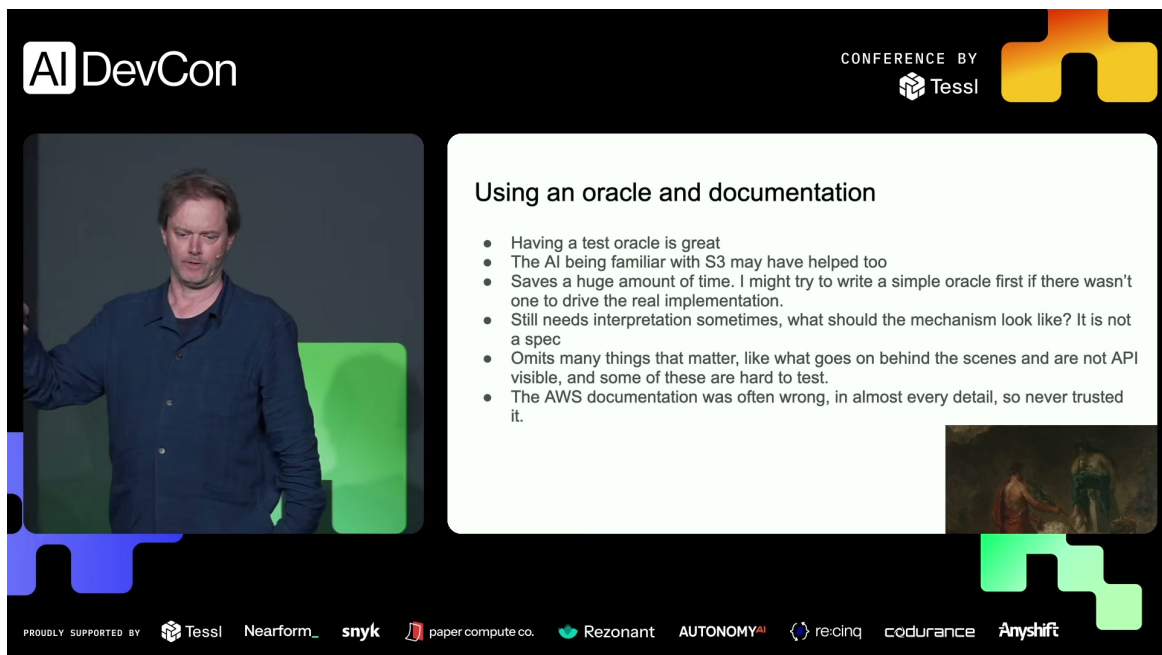


图 3: 可执行的 S3 行为提供强测试预言机；文档仍需通过实测校验，并由人解释机制边界。³

³视频来源：00:06:48–00:09:39；选帧 00:09:08。

2.5 避免同一逻辑运行两遍：预言机独立性

问答环节进一步暴露了自建测试预言机的风险。若 AI 按同一种算法实现生产系统，又用同一种算法实现预言机，测试只是把同一逻辑运行两次。相同输出最多证明两份代码一致，无法证明共同假设正确。

独立性可以从三个方向增强。其一，让参考模型采用更简单、不同的算法，即使速度很慢也可以接受。其二，把模型放在独立目录或仓库中，冻结接口和核心规则，避免跟随被测实现同步修改。其三，加入来自外部系统、人工构造边界和历史故障的多源样本，使一条逻辑链无法控制全部期望。

演讲者明确承认，外部 S3 让该项目较容易获得独立参照；他遇到的部分模型测试仍会引发“究竟测到了什么”的疑问。这个限制十分重要：测试预言机是质量证据的一层，其可信度取决于来源差异、可审查性和稳定性。

2.6 本章小结

可信测试先要回答“期望从哪里来”。外部系统或简单参考实现可以建立行为基线；最终一致性要求有边界的重试策略，后台不可见行为要求额外观测，文档与实测冲突需要人工裁决。最关键的约束是预言机与被测实现保持逻辑独立。下一章将在这条基线上扩展测试空间，让失败成为发现未知风险的信号。

3 把测试变成发现工具

高价值测试围绕未知风险扩展搜索空间。团队应优先探索边界条件，组合多种生成与重复运行方法，保持反馈快速稳定，并把每次失败转化为新的测试需求或机制修复。

3.1 边界条件为何比覆盖数字更有信息量

边界条件 (edge case) 位于输入范围、状态组合或容量边缘，例如零长度、单元素、超过上限的长度以及特殊编码。它并不泛指所有罕见事件。边界之所以有价值，是因为实现通常在“从无到有”“从一个到多个”“合法到非法”的转换处切换分支，遗漏也更容易集中在那里。

演讲者建议让 AI 生成零长度、长度为一、超限长度等常见边界，同时由人提出值得怀疑的区域。单纯要求“增加覆盖率”偶尔也会碰到边界；更高效的方式是说明状态结构、协议限制和曾经失败的机制，让 AI 在有意义的搜索空间中扩展用例。

3.2 发现 S3 可重复 500 错误所代表的证据

该项目在强化解析数据类型时，AI 写出一个针对 S3 的测试：通过 XML 转义写入零字节 %00，触发了可重复的 HTTP 500 错误。之后，分段上传校验和的另一个边界也触发了可重复 500 错误。演讲者把这视为测试集开始深入探索异常空间的信号。

这类发现的证据价值来自四个条件：输入具体、结果可重复、外部参照独立、失败落在非典型但合法可构造的路径。发现外部系统缺陷并不能证明本实现整体正确，也不能量化测试空间已经覆盖多少；它至少说明测试不再只验证最平凡的成功路径。

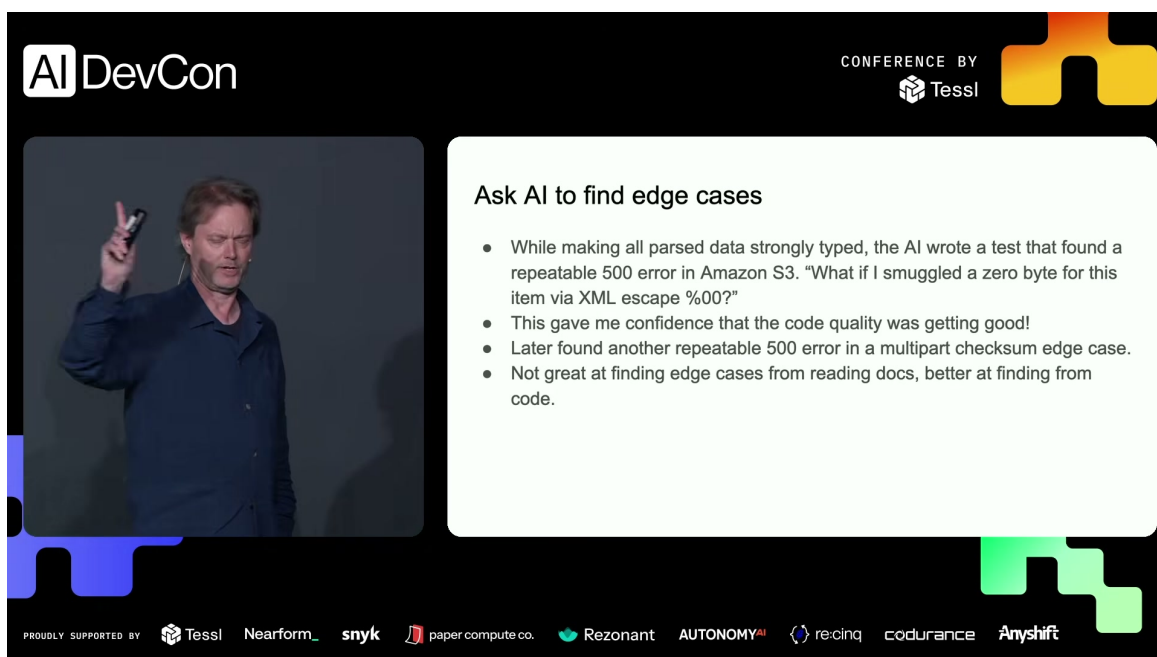


图 4: AI 从强类型解析代码中发现可重复的 S3 500 错误，显示代码结构可以成为边界条件的搜索空间。⁴

3.3 人与 AI 的测试分工：人提出可疑区域，AI 扩展和执行

演讲中的分工经验很具体：AI 在阅读正在实现的代码、枚举常见边界和批量生成测试方面表现较好；面对长篇文档，它较难主动识别哪些细节值得怀疑。人则能结合经验阅读文档，提出“这条描述真的成立吗”“这里是否缺少状态转换”等问题，再让 AI 生成实验。

因此，人的工作重点是选择搜索方向、解释失败和判断证据是否充分；AI 的优势是低成本扩展输入、重复执行并整理结果。这种分工仍是该项目的经验观察，并非所有模型与任务上的固定能力边界。团队应记录哪些提示产生了有效缺陷、哪些方法只增加了噪声，再据此调整下一轮测试策略。

一次故障应推动两项工作：先修复触发故障的机制并保留回归测试，再追问同类错误还能在哪些输入、状态或模块边界出现。前者防止复发，后者扩大未知区域的探测面。

3.4 不稳定测试会污染反馈回路

不稳定测试（flaky test）是在被测代码和输入没有实质变化时仍可能交替通过或失败的测试。它不同于能够稳定重现的确定性失败。对 AI 代理而言，不稳定信号尤其危险：代理可能根据训练数据中的常见做法忽略失败，也可能误以为外部服务改变了行为，继而反复修改实现去追逐噪声。

演讲案例中的部分波动来自 S3 授权状态的最终一致性。团队需要区分三种情况：已知收敛语义应采用有边界的重试；真实竞态需要修复实现；测试自身依赖未控制的时间、机器或顺序时，需要修复测试。把所有失败统一加重试，会掩盖后两类问题。

演讲者采用的硬规则是立即治理不稳定测试，并让测试尽可能快、频繁运行。该项目当时约 5000 个测试可在两分钟左右完成，还会跨机器执行通宵重复运行以捕捉低频失败。“两分钟”只是该案例可

⁴视频来源：00:10:02-00:12:57；选帧 00:12:38。

接受的反馈预算，其他项目应按提交频率、计算成本和风险设置自己的上限。

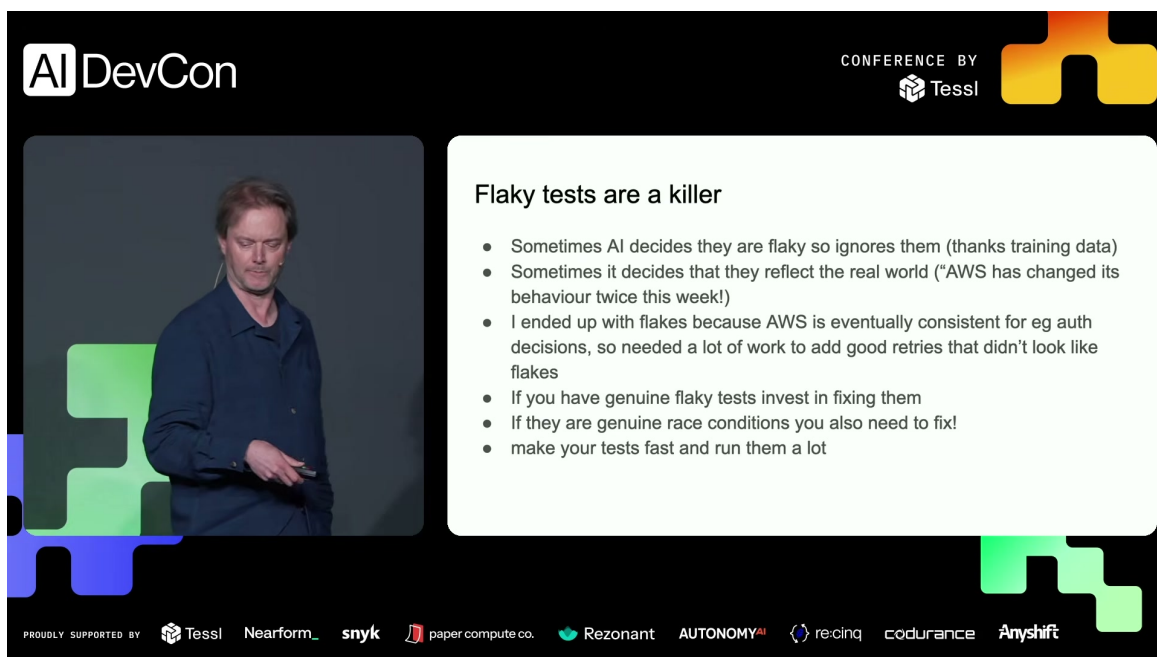


图 5: 不稳定测试会教会代理忽略失败；快速运行、频繁复现与立即修复共同保护反馈回路。⁵

不稳定测试不是可以长期容忍的背景噪声。它会降低所有失败信号的可信度，并诱导自动化代理修改正确代码、放宽断言或无限增加重试。

3.5 模糊测试、基于性质的测试与重复运行

模糊测试（fuzz testing）通过自动生成或变异大量输入，寻找崩溃、异常状态或未预期行为；它仍需要明确的失败判据，不能只是随机调用接口。基于性质的测试（property-based testing）先声明对一类输入应普遍成立的性质，再自动生成多组输入检验它，例如“序列化后再反序列化应恢复等价对象”。它不要求为每个输入手写一个期望输出。

演讲者报告，这两类方法都在项目中发现过问题。重复运行则回答另一个问题：同一条件下结果是否稳定。三者形成互补关系：模糊测试扩大输入多样性，基于性质的测试扩大规则验证范围，重复运行扩大时序与环境采样。它们无法自动替代测试预言机、风险判断或故障解释。

可执行的采用顺序是：从最近一次真实故障提炼性质或输入生成策略；为运行设置资源上限和可复现种子；保存最小失败样本；将已确认故障转为稳定回归测试；再观察新增方法是否发现了原有套件看不到的错误。方法数量只有在带来新证据时才有价值。

3.6 本章小结

测试应成为不确定性探针。边界条件把注意力集中在状态转换处，可重复的外部异常提供强证据；人负责提出可疑区域和解释风险，AI 负责扩展与执行；不稳定测试必须及时分类和治理；模糊测试、基于性质的测试与重复运行从不同维度扩大搜索空间。下一章将处理这些测试仍看不到的内部状态、时序和结构风险。

⁵ 视频来源：00:13:12–00:14:55；选帧 00:14:53。

4 测试边界之外：让内部行为可观测、可约束

测试能够稳定回答一个重要问题：给定输入后，公开接口是否产生了预期结果。大型并发系统还包含另一类问题，例如一次删除究竟何时在后台完成、锁竞争为何只在夜间压力运行中出现、权限检查与实际使用之间是否留下时序窗口、某次优化是否真的改善了无追踪条件下的性能。这些属性若没有被系统主动暴露，测试即使全部通过，也只能说明观察窗口内没有看到失败。

本章的结论是：工程团队需要把原本隐蔽的状态转成证据。管理接口扩大可询问的范围，追踪保存一次执行的因果路径，类型系统提前排除一类非法状态，独立安全审查从不同视角寻找遗漏。四者与行为测试共同组成分层反馈系统。

4.1 黑盒测试的盲区：竞态、性能、安全与架构

黑盒测试依赖可输入、可观察、可断言的边界。若一个属性没有可测信号，测试框架无法凭空判断它。演讲列出的盲区可以分为四类：

- **时序盲区**：竞态条件（race condition）指结果依赖多个并发操作不可控的先后顺序。大量快速重复测试有机会撞到某个时序，却无法证明所有时序安全。
- **性能盲区**：回归测试可以发现明显变慢，仍需基准、负载模型与正确的比较对象才能回答“性能是否足够好”。
- **安全盲区**：已有用例通常验证预想的合法路径，攻击面恰好集中在开发者尚未想到的组合、权限边界与数据流。
- **架构盲区**：模块是否承担过多职责、依赖方向是否失控、接口是否泄漏内部细节，属于结构判断，单个输入输出断言无法充分表达。

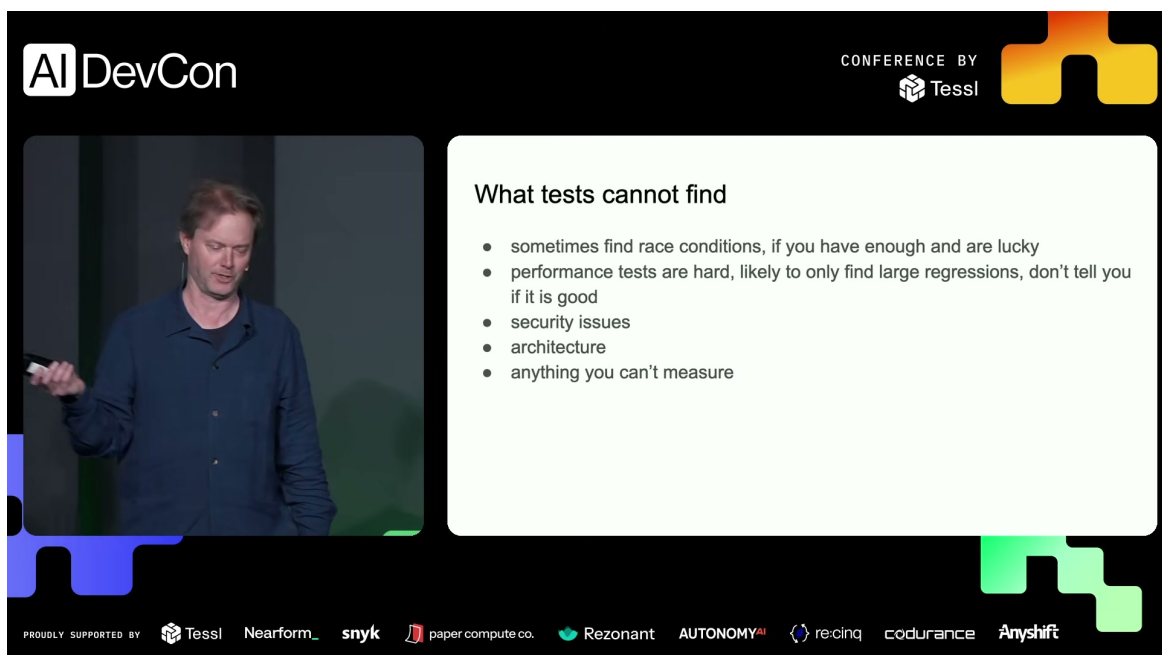


图 6: 讲者列出的测试盲区：竞态、性能、安全、架构与尚不可测量的属性。⁶

⁶视频帧来源：00:16:47–00:17:31；选帧 00:17:08。

这里需要保持一个边界：测试偶尔发现竞态，并不代表测试是竞态验证的充分工具；安全缺陷可以被回归测试固定下来，也不代表回归测试能够主动发现未知攻击路径。恰当策略是先识别“当前看不见什么”，再为该属性增加信号或约束。

4.2 管理接口和后台接口扩大可测试表面

可观测性（observability）是依据日志、指标、追踪和诊断接口推断内部状态与事件过程的能力。它的关键并非日志数量，而是信号能否回答工程问题。对于对象存储系统，公开 API 可以回答对象能否读写，却未必能回答后台回收何时发生、队列是否积压、某个副本处于何种阶段。

讲者回顾项目时指出，早期过度聚焦兼容 S3 的公开 API，管理、报告和后台接口建设不足。单元测试虽然触及内部代码，工程师与代理仍缺少可交互、可查询的稳定表面。这一经验可以转成三步设计原则：

1. 先列出影响正确性的内部状态，例如任务阶段、重试次数、锁等待、对象生命周期和授权结果；
2. 为诊断目的提供受控的只读管理接口，输出具有明确字段与语义的状态；
3. 让测试和调试工具通过这些接口提出问题，并保留访问控制与敏感信息边界。

管理接口也会带来风险。临时诊断入口可能演变成未经保护的控制面；暴露过多内部细节会固化实现并增加兼容负担。接口需要最小权限、稳定语义和明确的非生产用途说明。可测试性扩大的是证据表面，不能以牺牲安全边界为代价。

4.3 追踪如何把偶发故障变成 AI 可复现的证据

追踪（tracing）记录一次请求跨组件经过的步骤、时序和上下文。离散日志像散落的目击证词，追踪记录则把相关事件串成一条可查询路径。对于只在特定并发顺序下出现的超时或竞态，这条路径能把“昨夜偶尔失败”转成具体的事件序列。

演讲中的做法很朴素：先让 AI 构建一个手工维护的早期追踪框架，无需立即接入生产级平台；夜间重复测试一旦捕获罕见故障，就把对应追踪交给 AI，要求其复现同一路径并验证修复。此时 AI 的搜索空间由任意猜测收窄为可观察的时序、组件和状态。

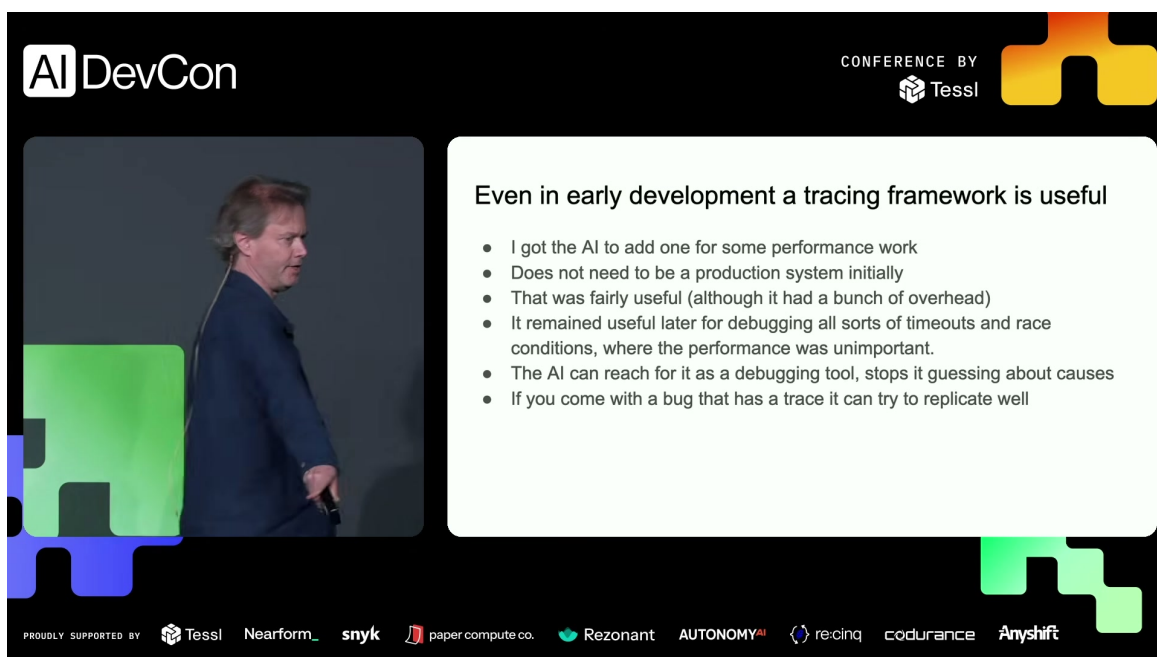


图 7: 早期追踪框架无需达到生产级，也能为超时与竞态调试提供因果线索。⁷

可执行的故障闭环可写成普通工程步骤：捕获失败，保存追踪上下文，重放或构造最小复现，修复后用同一证据验证，再把确定性失败固化为回归测试。追踪承担“发生了什么”的动态观测，回归测试承担“以后不能再发生”的行为约束。

追踪本身会改变系统。讲者明确提到其实现存在开销，因而追踪结果适合定位大热点，不能直接等同于无追踪运行的真实性能。采样、序列化与上下文传播也可能扰动原有时序。团队应分别记录诊断配置与基准配置，并在关闭或降低追踪开销后复测性能结论。

4.4 性能工程：测量、试错、丢弃无效优化

性能优化适合被视为实验循环。代理根据追踪提出的优化只是待检验假设：建立可重复基准，实施一个受控改动，比较吞吐、延迟与资源使用；结果无改善或出现退化时，直接回滚该改动。生成修改的成本很低，保留无效复杂度的长期成本很高。

⁷ 视频帧来源：00:18:00–00:20:31；选帧 00:19:38。

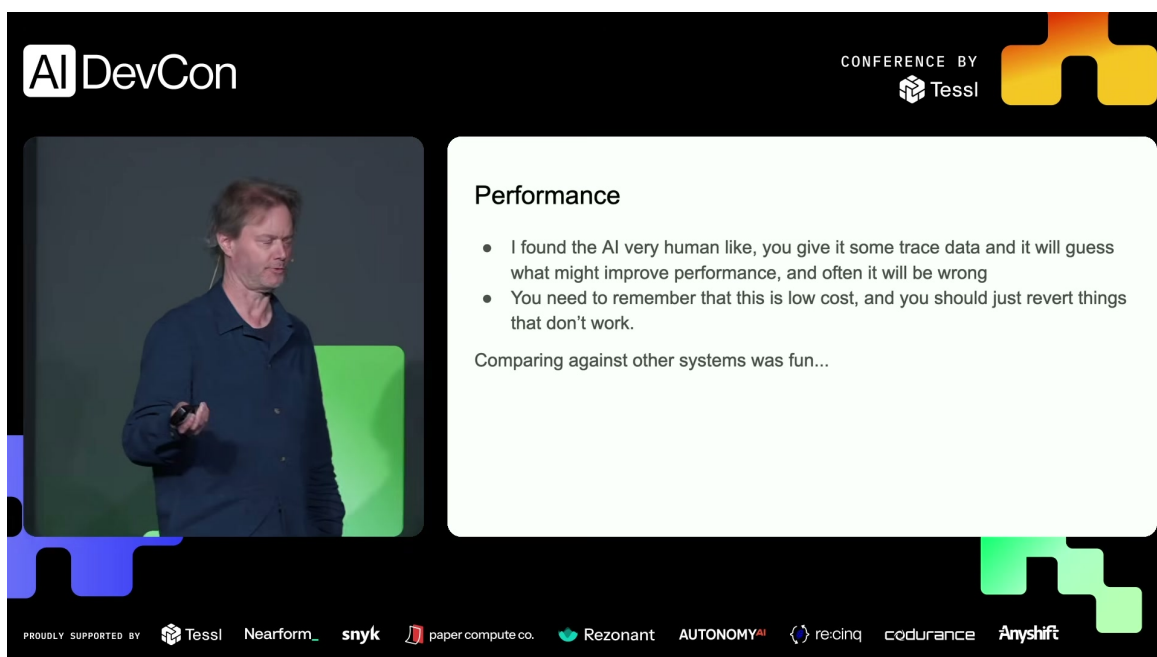


图 8: 性能建议需要经过实测；无效方案应回滚并换一个假设。⁸

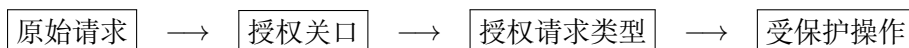
演讲给出一个说明比较基线为何重要的例子：项目的读取性能可以与另一种 S3 实现接近，写入性能却明显较慢。进一步检查发现，对方实现的代码注释说明写入时没有执行同步落盘。两者承担的持久性语义不同，直接追平数字会诱导错误优化。性能比较必须同时核对工作负载、正确性保证、持久化策略和观测开销。

性能数字只有在语义等价时才可比较。若一方省略同步落盘、校验或复制步骤，速度差异可能来自工作量差异，而非实现效率。

4.5 类型系统作为结构性安全约束

追踪可以展示一次权限路径发生了什么，类型系统则可以限制哪些路径有资格存在。演讲中的问题是检查时刻到使用时刻问题（time-of-check to time-of-use）：请求先通过权限检查，实际使用前状态发生变化，或者不同模块重复检查且语义不一致，先前结论便可能失效。

讲者最终采用授权请求类型（authorized request type）：原始请求只能经过授权关口后被转换为该类型，受保护操作只接受这种类型。于是调用链形成清晰结构：



这种静态约束与动态观测互补。追踪能说明某次运行是否走错路径；类型约束让未授权请求难以被构造成后续函数可接受的值。它也有明确限制：类型系统只能执行已编码的规则，无法证明权限策略完整、授权数据及时或运行时依赖可信。接口审查仍需检查模块边界是否合理。

⁸视频帧来源：00:20:27-00:21:40；选帧 00:21:08。

4.6 独立安全审查弥补日常代码审查遗漏

安全审查需要独立于功能实现与常规测试的观察角度。讲者把安全工具发现提交进代码库，要求 AI 逐项复核，并在每次修复说明中记录修复方式与对应提交链接。把已确认问题固化为回归测试，则是本文基于该流程补充的工程建议。积累一批问题后，再举行复盘，追问哪些结构、测试或开发规则本可更早阻止这些问题。

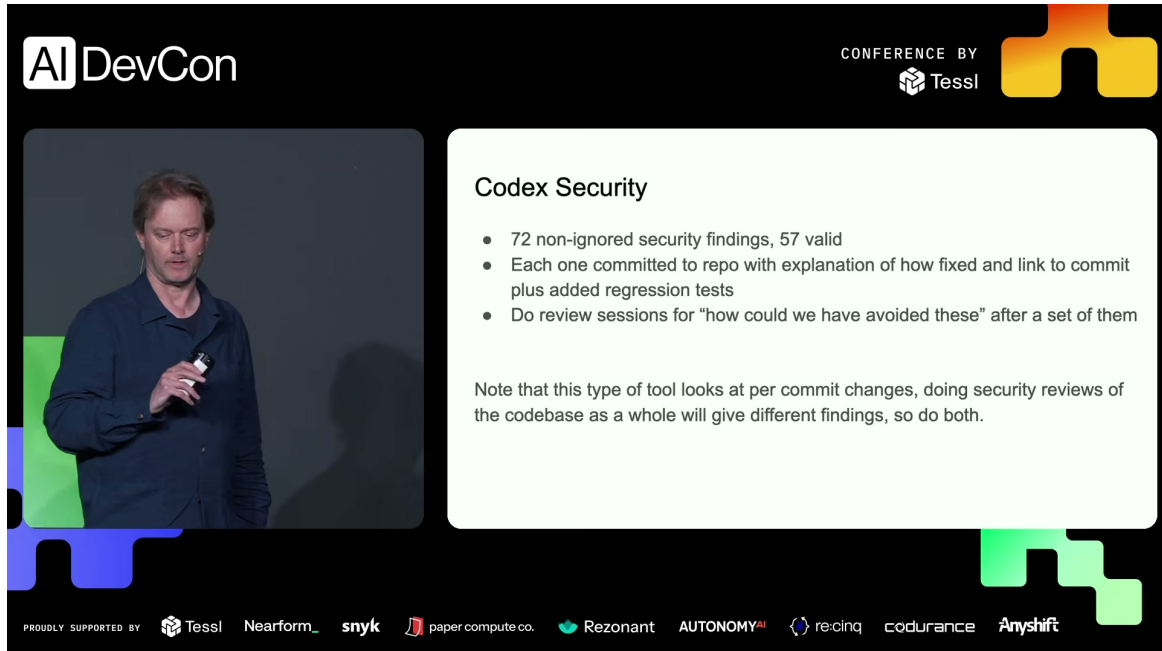


图 9: 该项目的安全审查案例：72 项未忽略发现中，复核后 57 项被判定有效。⁹

这些数字不能泛化为工具的通用准确率，也不能把“有效发现”等同于全部属于严重漏洞。它们证明的范围更窄：在该项目中，独立审查找到了日常 AI 代码审查迭代遗漏的重要问题。提交级扫描观察局部变化，全库审查观察跨模块关系和累积风险，两者覆盖面不同，均需要人工复核。

4.7 本章小结

测试边界之外的工程工作可以归纳为“暴露、记录、排除、复核”：管理接口暴露内部状态，追踪记录具体执行路径，类型系统排除一类非法状态，独立安全审查复核实现者与常规流程的盲区。每一种机制回答不同问题。可观测性提供证据，结构约束减少可构造错误，审查寻找尚未进入测试空间的风险；它们共同让 AI 的修改受到可复核证据约束。

5 人的职责：经营质量反馈回路

AI 能快速生成实现、测试、追踪工具和审查意见，质量责任仍然落在人身上。原因很直接：自动化只能优化已给出的信号，而“什么值得测量”“何种失败可接受”“何时应停止补丁并重构”本身是工程判断。人的核心工作由逐行生产代码转向设计、观察和校正反馈回路。

⁹视频帧来源：00:22:54–00:23:57；选帧 00:23:08。数字属于该项目的经验记录。

5.1 人在回路中：质量责任不能外包

人在回路中（human in the loop）指人在自动化流程内持续定义目标、检查信号、判断风险并决定下一步。它不要求每次生成后逐行阅读 35 万行代码，也不等于偶尔按下批准按钮。有效参与至少包含四项职责：

1. 定义“足够好”的质量模型，包括行为、安全、性能、可维护性和架构边界；
2. 观察失败模式，识别代理是否忽略不稳定测试、修改测试迎合实现或滥加重试；
3. 为未知风险选择新的信号，例如管理接口、追踪、类型约束或独立审查；
4. 在局部修补开始掩盖结构问题时，决定重构范围和停止条件。

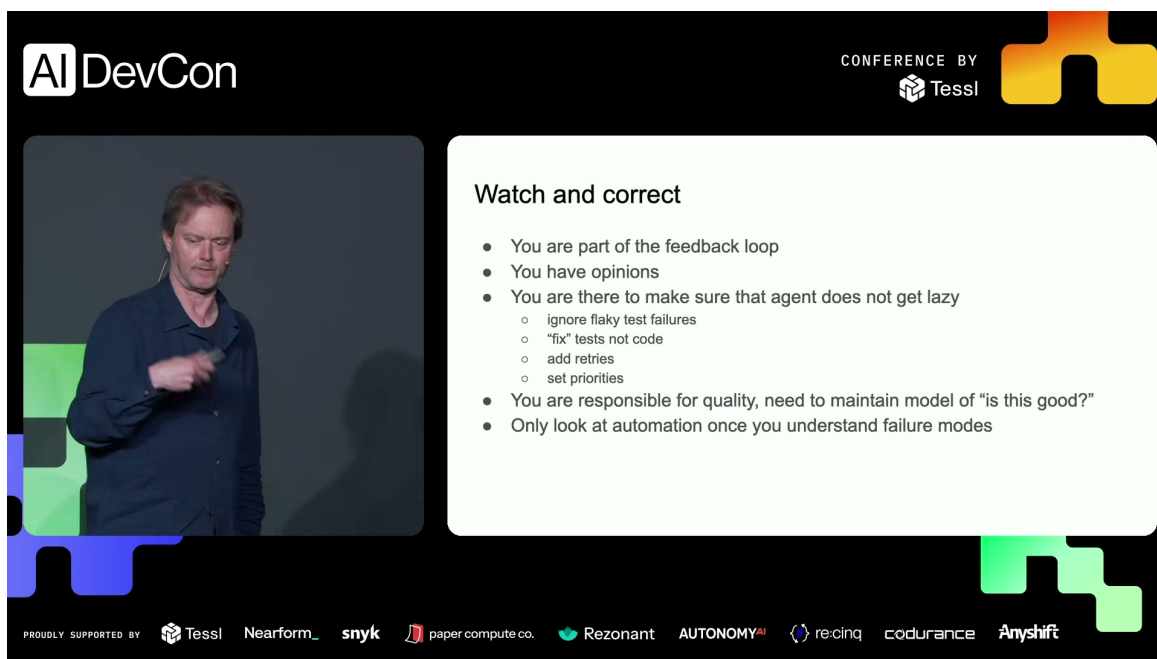


图 10: 人在反馈回路中维护质量判断，并纠正代理对失败信号的投机处理。¹⁰

“让 AI 保持诚实”在本文中是一种工程隐喻：代理的修改必须接受独立基线、可观察证据和人的风险判断。这里没有人格化或道德判断；所谓“诚实”就是无法用修改断言、忽略偶发失败或增加无依据重试来绕过质量目标。

过早自动化会隐藏学习机会。讲者有意保留人工参与，是为了先理解系统会怎样失败，再决定哪些流程可以稳定自动化。若团队尚不理解失败模式，自动化只会更快地重复错误策略。

5.2 重构是收敛过程的一部分

大规模 AI 编码容易制造“一次生成即可完成”的错觉。演讲中的提交图呈现了相反过程：新增与删除代码同时维持高位，某一周约新增 12 万行、删除 7.5 万行；项目还曾出现一个约 4.3 万行的单文件，需要专门重构。上述数字属于该项目的阶段记录，说明系统通过持续重写而收敛，不能作为其他项目的规模阈值。

¹⁰ 视频帧来源：00:24:02–00:25:12；选帧 00:24:38。

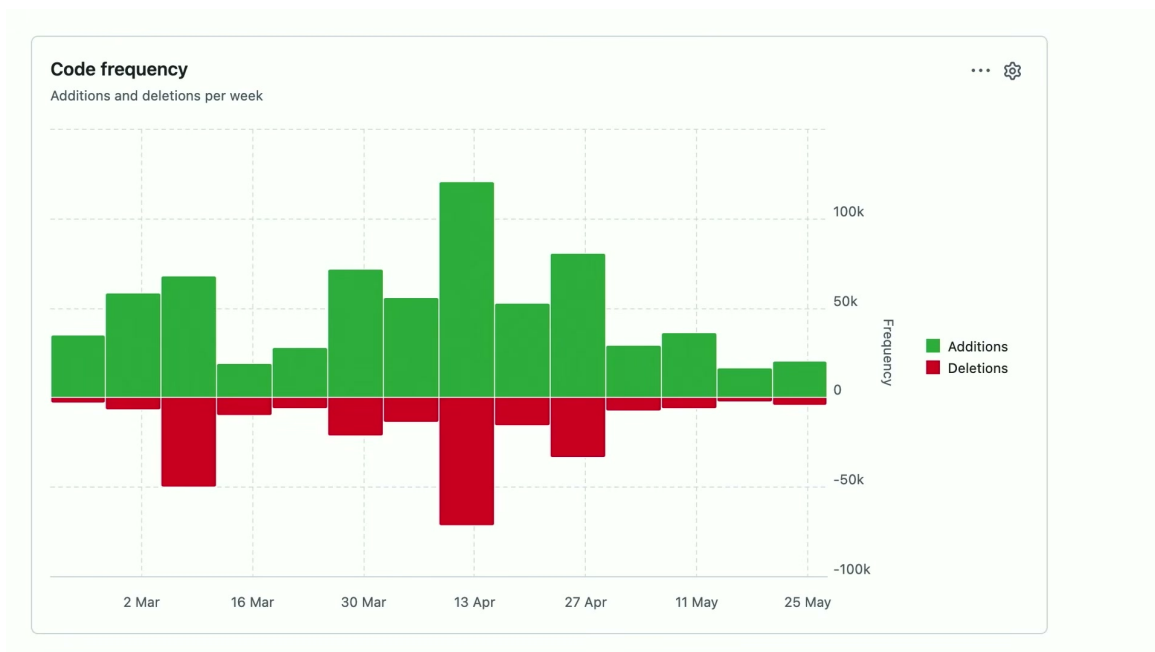


图 11: 每周新增与删除代码并行发生，显示实现通过反复重写逐步收敛。¹¹

重构在反馈回路中承担两种功能。第一，它把已经学到的行为和边界重新编码为更深的模块接口、类型和职责划分。第二，它清除探索阶段积累的重复逻辑、临时兼容层和过大文件，使后续代理更容易在有限上下文中正确修改系统。

重构仍需行为基线保护。缺少稳定测试预言机与回归测试时，大范围重写会同时改变结构和语义，失败后难以归因。可行顺序是先固定外部行为与关键内部不变量，再拆分结构，每一步运行快速测试并检查追踪或性能信号。

5.3 测试从验收清单转为不确定性探针

演讲结尾强调，测试是发现工具。固定验收清单回答已知问题；不确定性探针主动寻找团队尚未理解的区域。实践中，团队应从失败、怀疑和结构变化出发扩展测试：某种输入曾触发异常，就系统化搜索相邻边界；某段并发逻辑令人不安，就增加重复运行和追踪；某个安全发现成立，就补回归测试并追问是否存在同类结构。

¹¹ 视频帧来源：00:25:18–00:25:44；选帧 00:25:30。讲者口述的高峰约为新增 12 万行、删除 7.5 万行。

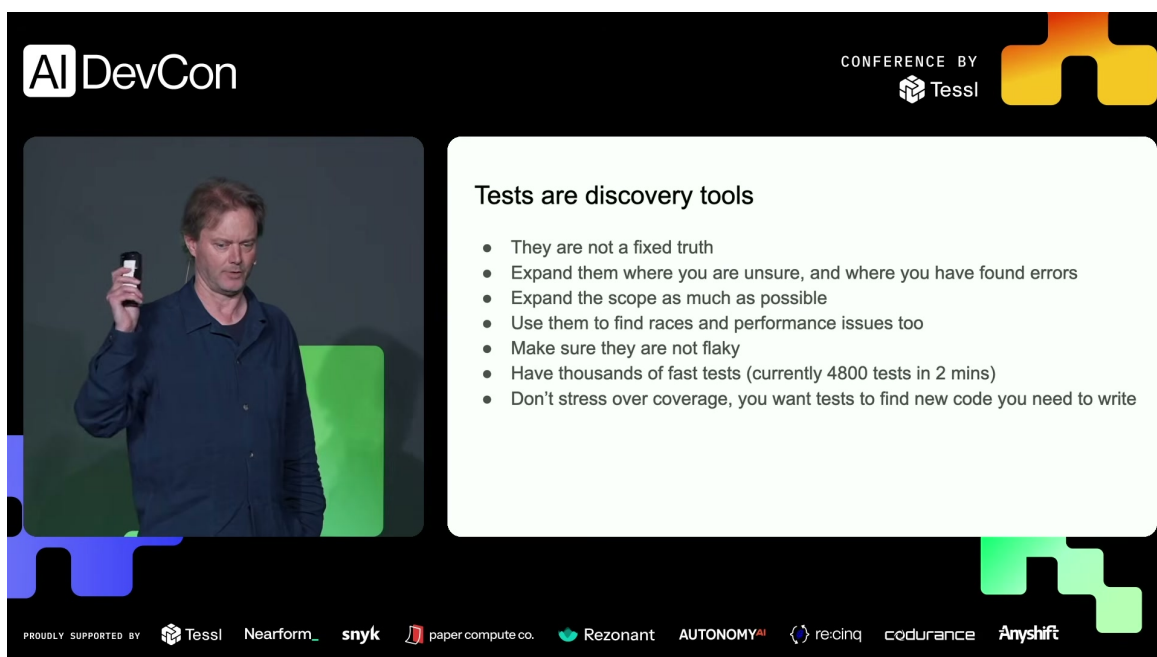


图 12: 讲者的总结：测试应沿不确定区域扩展，保持快速稳定，并用于发现下一步代码需求。¹²

这种工作方式还改变覆盖率的地位。覆盖率可以提示未触达代码，却不能替代风险判断。团队应优先追问：失败是否揭示新的状态空间？当前断言能否识别错误实现？内部信号能否解释原因？若答案不清楚，继续提高覆盖数字可能只会增加低价值用例。

5.4 可执行 workflow：基线—观测—约束—复核—重构

下面的五步 workflow 把前述职责落到日常迭代：

1. **基线**：先用独立测试预言机或已确认行为固定外部语义，明确重试与一致性边界。
2. **观测**：运行快速、多样、稳定的测试；对罕见故障保存追踪和管理接口状态。
3. **约束**：把重复出现的权限、状态和模块边界错误转成类型或架构约束。
4. **复核**：由独立安全审查与人检查发现，区分有效风险、工具误报和证据不足。
5. **重构**：当局部修补暴露结构问题时，保持行为基线，重组代码并重新运行整个反馈链。

每一次失败都应留下可复用产物：最小复现、回归测试、追踪样本、结构约束、审查记录或架构决策。若失败只被临时重试掩盖，反馈回路没有获得新知识。

人的不可替代职责是维护质量目标与证据标准：决定何时相信自动化结果，何时扩大观测面，何时把反复出现的问题提升为结构约束，以及何时投入重构。

5.5 本章小结

人的角色是经营反馈回路。自动化提高探索与修改速度，人负责设定质量模型、识别投机性修复、选择证据通道并推动结构收敛。重构属于正常工程过程，测试则持续探测不确定区域。每轮迭代应把

¹² 视频帧来源：00:26:23–00:27:12；选帧 00:27:08。幻灯片记录该项目当时约 4800 个测试、两分钟完成。

一次失败转成可复用的行为基线、观测信号或结构约束。

下一章将在这套日常迭代职责之上压缩全局框架：把行为基线、测试、内部观测、结构约束、独立审查与人的判断放入同一张反馈回路，并明确案例尚未覆盖的验证边界。

6 总结、限制与下一步

本文依据讲者贯穿演讲的实践线索，将可靠 AI 软件工程综合为六层反馈回路：独立行为基线、快速多样测试、内部观测、结构约束、独立审查和人的复核与重构。这是本文的教学归纳，并非讲者提出的固定六层模型。六层分别回答“预期是什么”“哪里可能错”“内部发生了什么”“哪些错误不应可构造”“还有哪些遗漏”“风险是否可接受”。任何单一指标都无法替代整套闭环。

6.1 六层反馈回路的概念压缩

层次	核心问题	主要产物
行为基线	外部可见结果应当是什么？	独立测试预言机、差分结果、明确的一致性边界
多样测试	哪些输入、状态和时序可能暴露错误？	快速稳定的回归、边界、模糊与基于性质的测试
内部观测	失败发生时，系统内部经历了什么？	管理接口、日志、指标、追踪记录与最小复现
结构约束	哪些非法状态或数据流可以提前排除？	类型约束、授权关口、清晰模块接口
独立审查	实现者与常规流程遗漏了什么？	安全发现、全库审查、复核结论与回归测试
人的复核与重构	证据是否足够，风险是否可接受，结构是否需要重做？	质量判断、优先级、重构计划与停止条件

表 1: 根据演讲整理的六层反馈回路。

反馈并非单向流水线。安全审查发现权限路径缺陷后，可以新增回归测试、追踪字段和授权请求类型；追踪暴露竞态后，可以增加重复测试并调整模块边界；重构又会触发行为基线与性能基准的重新验证。闭环的价值来自证据在层次之间流动。

6.2 何时选择测试、追踪、类型约束或安全审查

工具选择应由问题性质决定：

- 已知输入输出关系需要长期固定时，选择行为测试；若有外部实现或简单模型，优先使用独立测试预言机。
- 故障罕见、跨组件且难以复现时，选择追踪并保存时序上下文；诊断后再固化回归测试。
- 错误来自某类值或权限路径根本不应进入后续模块时，选择类型约束和接口重构。
- 风险涉及未知攻击路径、跨模块数据流或累积架构问题时，选择独立安全审查与全库审查。
- 性能问题需要基准、等价语义和反复实验；追踪负责定位，最终结论由低扰动测量决定。

隐藏依赖也必须进入判断：外部服务行为会变化，追踪会扰动性能，类型只能约束被编码的规则，扫描会产生需复核的发现，人也可能持有错误的质量模型。多层证据的意义，就是避免任一依赖成为未经检查的单点真相。

6.3 演讲结尾的实质性观点：持续重构与主动探索不确定性

讲者的收束包含两条互相支撑的观点。第一，大系统通过持续重构收敛，团队无需期待一次生成得到最终结构。新增与删除代码同时发生，说明实现不断吸收测试、观测与审查带来的知识。第二，测试是发现工具，应沿失败、怀疑和未知区域扩展，并保持足够快、足够稳定，使代理能够频繁使用它们。

这两条观点共同修正了“测试通过即完成”的工作模型：通过表示当前问题在当前证据下得到回答；新的规模、状态组合、部署方式或安全视角仍会产生新问题。工程团队需要主动寻找这些问题，并允许结构随认识更新。

成熟的完成标准不是“再也没有失败”，而是关键风险已有独立基线、可复现证据、结构约束和明确负责人；新失败出现时，反馈回路能够稳定地产生下一步行动。

6.4 未完成工作：分布式验证与形式化验证

问答环节明确给出了项目边界：演讲时，多主机分布式阶段仍在实现，尚未完成验证；形式化验证（formal verification）也尚未执行。形式化验证使用数学化模型与证明方法检查系统是否满足明确性质，它适合处理某些状态转换、不变量和并发协议问题，仍依赖正确的模型与性质定义。

因此，本案例不能证明 AI 已经成功构建并验证完整分布式对象存储。更严谨的下一步是：先让多主机实现达到可运行状态，明确故障模型与关键不变量，再评估模型检验或证明工具能够覆盖哪些性质。演讲者也提到关于 AI 辅助形式化验证的报告并不一致，这一领域需要具体实践证据。

6.5 预言机独立性的开放问题

问答中的第二个实质问题触及测试体系的根基：若 AI 用同一套逻辑编写被测系统和测试预言机，两者可能只是把同一错误运行两遍，输出一致却没有增加可信度。外部 S3 在本案例中提供了较强独立性，自建预言机则需要刻意隔离。

可采取的措施包括：采用更简单的模型与不同算法路径；将模型固定在独立代码库或独立版本中；由不同人员或代理依据行为契约实现；对预言机本身加入示例、性质和人工审查。这里仍没有自动判据能够证明“足够独立”。团队必须说明两套实现共享了哪些需求、数据和生成上下文，并把同源逻辑风险作为开放审查项。

6.6 面向实践的检查清单

在交付或扩大 AI 自动化范围前，团队可以逐项回答：

1. 是否存在独立于被测实现的行为基线？若使用外部服务，其版本变化、最终一致性和不可见状态是否已记录？
2. 测试是否快速、稳定，并覆盖多种方法与高风险边界？不稳定测试是否立即治理？

3. 黑盒不可见的内部状态是否通过受控管理接口或追踪暴露？故障证据能否支持最小复现？
4. 可由类型和架构排除的错误是否已经结构化消除？授权关口与模块接口是否清晰？
5. 是否存在独立的安全审查、发现复核、回归测试，以及覆盖全库关系的整体审查？
6. 人是否持续审查失败信号、质量目标、风险接受和重构方向？自动化是否建立在已理解的失败模式上？

若任何一项只能回答“未知”，该未知就是下一轮工作入口。团队可以增加测试、观测或审查来收集证据；若问题来自反复出现的结构缺陷，则应进入类型或架构重构。检查清单的用途是揭示缺口，不能被机械勾选替代工程判断。

6.7 本章小结

全文的最终结论是：测试继续承担行为验证与不确定性探索；按照本文的六层归纳，可靠性来自多类反馈机制的共同作用。独立基线防止自我证明，快速多样测试扩大搜索空间，内部观测保存因果证据，类型与架构约束减少非法状态，独立审查寻找遗漏，人负责判断风险并推动重构。分布式阶段与形式化验证仍待实践，自建预言机的逻辑独立性也需持续审查。对团队最有价值的下一步，是找到检查清单中的第一个“未知”，为它建立可复核证据。